

IN1097 · Tópicos Avançados em Agentes Inteligentes 2

# Características de Datasets (Meta-atributos)

Aula 6

Profa. Juscimara G Avelino

# Onde paramos

- Separamos avaliação em **nível base** e avaliação em **meta-nível**
- Comparamos rankings preditos com o **ranking verdadeiro** usando correlação de ranks
- Medimos o efeito da recomendação com **curvas de perda** por número de testes e por tempo
- Em toda essa história, cada **dataset era uma instância** — mas nunca dissemos quais colunas descrevem um dataset
- **Como transformar um dataset inteiro em um vetor de números?**

# Objetivos

- Definir **meta-atributo** e justificar os quatro requisitos de um bom meta-atributo
- Reconhecer as **quatro famílias** de meta-atributos para classificação e o que cada uma enxerga
- Interpretar as medidas de **complexidade**
- Adaptar meta-atributos para **regressão, séries temporais e agrupamento**
- Derivar novos meta-atributos por **agregação, projeção e fatoração**, e selecioná-los
- Medir **similaridade entre datasets** — a base de quase todo sistema de MtL

# Roteiro da aula

## 1 O que é um meta-atributo

Por que precisamos deles e o que faz um ser bom

## 2 As quatro famílias

Simples/estatísticas/informação, modelo, performance, complexidade

## 3 Outras tarefas

Regressão, séries temporais e agrupamento

## 4 Engenharia de meta-atributos

Agregação, projeção, fatoração e seleção

## 5 Similaridade entre datasets

Onde tudo isso é efetivamente usado

## PARTE 1

# O que é um meta-atributo

Transformando um dataset inteiro em uma linha de tabela

# A pergunta que a meta-aprendizagem faz

Meta-aprendizagem busca aprender a relação entre a performance dos algoritmos e as características dos dados

- Para aprender essa relação, precisamos **medir** as características dos dados — são elas os **meta-atributos** (ou meta-características, meta-features)
- Meta-atributos são **medidas padronizadas que codificam numericamente** as propriedades de um conjunto de dados
- É o arcabouço de **Rice (1976)**: o espaço de problemas é descrito por atributos, e a partir deles se prediz o desempenho relativo dos algoritmos
- O que se prediz pode ser uma **classe** (melhor algoritmo), um **valor** (acurácia esperada) ou um **ranking**

# Concretamente: uma linha por dataset

O meta-dataset tem meta-atributos à esquerda e o meta-alvo à direita. Cada linha é um dataset inteiro comprimido em números.

| dataset      | n      | p  | c  | % discretos | H(y) | acc. stump | melhor algoritmo |
|--------------|--------|----|----|-------------|------|------------|------------------|
| iris         | 150    | 4  | 3  | 0,00        | 1,58 | 0,67       | C4.5             |
| letter       | 20.000 | 16 | 26 | 0,00        | 4,70 | 0,12       | IB1              |
| soybean      | 683    | 35 | 19 | 1,00        | 3,84 | 0,28       | NB               |
| novο dataset | 1.024  | 12 | 2  | 0,25        | 0,97 | 0,71       | ?                |

- O dataset **letter** tem 20.000 linhas, mas no meta-nível vira **uma única instância**
- Por isso meta-datasets são pequenos — dezenas ou centenas de linhas. Isso terá consequências.

# O que faz um bom meta-atributo

## 1 Poder discriminativo

Precisa ter relação intrínseca com o meta-alvo. Se o valor não muda quando o melhor algoritmo muda, ele não informa nada.

## 2 Uniformemente computável

Todas as meta-atributos devem ser computáveis independentemente do dataset.

## 3 Baixo custo computacional

Se extrair o meta-atributo custa mais que testar todos os algoritmos base, o sistema perdeu o propósito.

## 4 Dimensionalidade contida

Poucas instâncias no meta-nível somadas a muitos meta-atributos levam a sobreajuste.

# O outro lado: caracterizar algoritmos

Nosso foco é caracterizar **datasets**, mas descrever também os **algoritmos** permite generalizar para algoritmos nunca vistos. Hilario e Kalousis (2001) propõem quatro eixos:

| Eixo                  | O que descreve                                 | Exemplo                                     |
|-----------------------|------------------------------------------------|---------------------------------------------|
| Tipo de representação | Que tipo de dado o algoritmo consegue tratar   | NB lida bem com nominais; kNN com contínuos |
| Abordagem             | Estratégia de aprendizagem                     | lazy (kNN) vs. eager (árvore)               |
| Resiliência           | Sensibilidade a atributos irrelevantes e ruído | kNN sofre com atributos irrelevantes        |
| Praticidade           | Facilidade de ajustar hiperparâmetros          | RF funciona no default; SVM não             |

**Descrever o algoritmo é o que permite um sistema opinar sobre um algoritmo que nunca viu em nenhum dataset**

# Não existe conjunto universal

O melhor conjunto de meta-atributos depende da tarefa,  
dos datasets e dos algoritmos candidatos

- A **tarefa base** determina o que é possível medir: sem variável alvo, medidas de entropia da classe simplesmente não existem
- Os **algoritmos candidatos** determinam o que é útil medir: a proporção de atributos contínuos separa NB de kNN, mas não separa NB de uma árvore
- Por isso a aula percorre **classificação, regressão, séries temporais e agrupamento** separadamente
- E por isso **engenharia de meta-atributos** continua sendo trabalho de pesquisa ativo, não uma lista fechada

## PARTE 2

# As quatro famílias, em classificação

Medir direto, medir por um modelo, ou medir testando

# Quatro famílias, três formas de medir

- |   |                                                        |                                                                                          |
|---|--------------------------------------------------------|------------------------------------------------------------------------------------------|
| 1 | <b>Simples, estatísticas e de teoria da informação</b> | <b>Medida direta</b> sobre a tabela de dados — contagens, momentos, entropias            |
| 2 | <b>Baseadas em modelo</b>                              | <b>Medida indireta:</b> induz um modelo e descreve as propriedades dele                  |
| 3 | <b>Baseadas em performance</b>                         | <b>Medida indireta:</b> roda algoritmos rápidos e usa a performance como descrição       |
| 4 | <b>Conceito e complexidade</b>                         | Quantificam o <b>quão difícil</b> é o problema: sobreposição, rugosidade, separabilidade |

# Meta-atributos simples

## Contagens diretas

- Número de exemplos,  $n$
- Número de atributos,  $p$
- Número de classes,  $c$
- Proporção de atributos discretos
- Proporção de valores faltantes de  $x_i$
- Proporção de outliers de  $x_i$

## Variantes e razões

- Em vez de  $n$ , usa-se  $\log(n)$  — a escala importa mais que o valor
- $n / c$  — exemplos por classe: densidade dos dados
- $n / p$  — exemplos por dimensão
- Estão entre os meta-atributos mais usados desde Rendell et al. (1987) e Michie et al. (1994)

# Por que a razão diz mais que o valor bruto

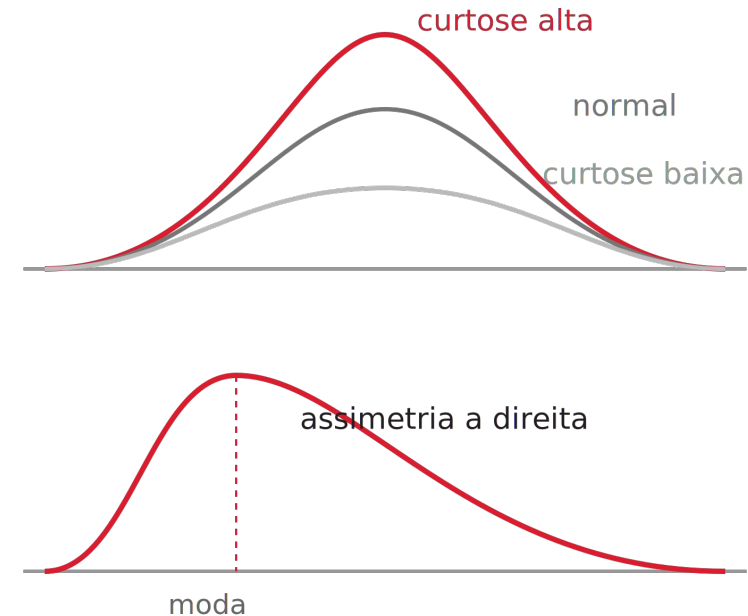
| dataset    | n      | p     | c  | n / c | n / p | leitura                                               |
|------------|--------|-------|----|-------|-------|-------------------------------------------------------|
| iris       | 150    | 4     | 3  | 50    | 37,5  | pequeno, mas denso e de baixa dimensão                |
| letter     | 20.000 | 16    | 26 | 769   | 1.250 | muitos exemplos por classe: fácil de povoar           |
| microarray | 72     | 7.129 | 2  | 36    | 0,01  | <b>n/p muito menor que 1: dimensionalidade domina</b> |

- Os três têm valores de  $n$  muito diferentes — mas é  $n/p$  que antecipa qual vai quebrar métodos baseados em distância
- Meta-atributos derivados por razão são baratos e frequentemente mais discriminativos que os originais

# Estatísticas descritivas de uma variável

- Aplicam-se tipicamente a **atributos numéricos** e descrevem o formato da distribuição
- **Assimetria** de  $x_i$  — a cauda puxa para um lado?
- **Curtose** de  $x_i$  — a distribuição é concentrada ou achatada?
- **Probabilidade da classe**  $p(y \square)$  — proporção de cada classe
- Alguns algoritmos assumem normalidade implicitamente; assimetria e curtose medem o quanto essa suposição é violada

**Por que isso prediz algoritmo?** Distribuições muito assimétricas podem ser menos adequadas a métodos com hipóteses distribucionais, como **LDA** e **Gaussian Naive Bayes**, enquanto **árvores de decisão** são, em geral, pouco sensíveis à forma da distribuição dos atributos.



# Estatísticas de relação entre variáveis

- **Correlação** entre  $x_i$  e  $x_j$ ,  $\rho(x_i, x_j)$  — e **covariância** entre eles
- **Concentração** entre pares de atributos nominais (Kalousis e Hilario, 2001)
- Todas estimam a **interdependência entre variáveis**: quanto os atributos apresentam informação redundante ou dependência entre si.

**Onde isso vira previsão:** correlação média alta entre atributos sugere maior redundância. Alta dependência entre atributos pode violar a hipótese de independência condicional do **Naive Bayes** e reduzir seu desempenho. Por outro lado, pode favorecer o uso de **PCA** antes da classificação, especialmente em algoritmos sensíveis à dimensionalidade e à correlação entre atributos.

- Note que uma correlação existe para **cada par** de atributos — o número de valores varia de dataset para dataset

# Entropia e informação mútua

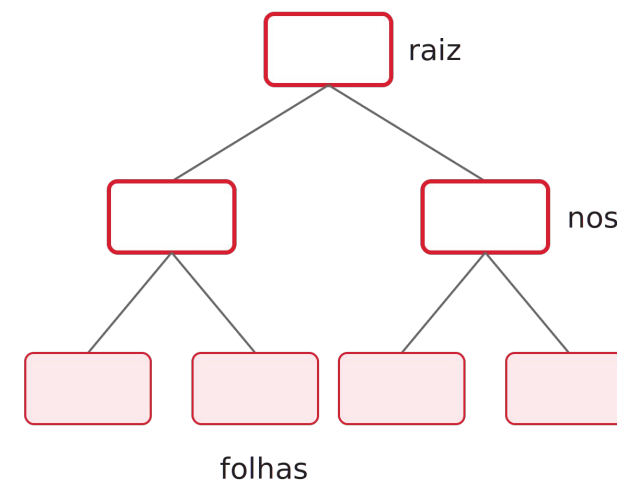
Tipicamente associadas a **atributos nominais** — medem incerteza em bits.

| Medida                      | Notação                            | Como interpretar                                    |
|-----------------------------|------------------------------------|-----------------------------------------------------|
| Entropia do atributo        | $H(x_i)$                           | quão espalhados são os valores do atributo          |
| <b>Entropia da classe</b>   | $H(y)$                             | <b>dificuldade da tarefa e desbalanceamento</b>     |
| Informação mútua            | $MI(x_i, y)$                       | quanto o atributo $x_i$ já revela da classe         |
| Dimensionalidade intrínseca | $H(y) / MI(x_i, y)$                | quantos atributos "equivalentes" seriam necessários |
| Razão ruído-sinal           | $[H(y) - MI(x_i, y)] / MI(x_i, y)$ | quanto do atributo é informação irrelevante         |

**Exemplo:** problema binário perfeitamente balanceado  $\rightarrow H(y) = 1$  bit. Se a proporção for 90/10,  $H(y) \approx 0,47$  bit. A entropia baixa avisa que a acurácia é uma métrica enganosa nesse dataset — e isso muda qual algoritmo vence.

# Meta-atributos baseados em modelo

- Induz-se um modelo do dataset e descrevem-se as **propriedades desse modelo**, não dos dados
- Caracterização **indireta**: os dados falam através da forma da árvore
- Básicos: **número de nós**, **número de folhas**, **comprimento dos ramos**
- Derivados: nós por atributo, folhas por classe, concordância entre folhas
- O modelo precisa ser **rápido de induzir** e **relacionado aos candidatos**



*4 folhas, 3 nos internos, profundidade 2*

**Leitura:** árvore rasa e pequena indica conceito simples e separável por eixos. Árvore enorme e profunda indica fronteira irregular, com provável vantagem de métodos não lineares.

# Landmarkers

Em vez de descrever o dataset, deixe alguns algoritmos rápidos o descreverem por você

Cada landmarker é a performance de um algoritmo simples e barato. O vetor dessas performances vira a descrição do dataset. Pfahringer et al. (2000) sugerem:

| Landmarker                          | O que ele revela sobre os dados                                      |
|-------------------------------------|----------------------------------------------------------------------|
| 1NN                                 | esparsidade e dispersão, especialmente perto da fronteira de decisão |
| Árvore de decisão ou decision stump | separabilidade por cortes paralelos aos eixos                        |
| Discriminante linear                | separabilidade linear                                                |
| Naive Bayes                         | independência entre atributos                                        |

- Vão um passo além dos baseados em modelo: representam a **performance** de um modelo, não suas propriedades
- Exemplo: para estimar como se sai uma árvore completa, usa-se como landmarker uma **decision stump** — a árvore de um nó só

# Três variações

## Relativos

*diferença em vez de valor*

- Baseados na **diferença ou razão** entre a performance de dois algoritmos
- Eliminam o efeito "dataset fácil": só a vantagem relativa sobrevive

## Por subamostragem

*o algoritmo real, em pouco dado*

- Roda-se o **próprio algoritmo** candidato numa **amostra** pequena
- Uma sequência ordenada de amostras forma uma **curva de aprendizado parcial**
- O meta-modelo usa o **formato da curva**, não só o valor final

## Múltiplos

*um vetor de sondas*

- Nada impede associar **vários landmarks** ao mesmo dataset
- Viram um vetor de performances — barato, uniforme e sempre computável

# Conceito e complexidade

Este grupo tem um objetivo único: **quantificar o quão difícil é o problema de classificação**. São três perguntas diferentes:

## Rugosidade

*o conceito é liso?*

- Vizinhos no espaço de entrada têm rótulos diferentes?
- **Medidas:** variação de conceito, não-linearidade do classificador linear e do 1NN

## Sobreposição

*os atributos separam?*

- As classes ocupam faixas distintas em algum atributo?
- **Medidas:** F1, F2, F3 e F4

## Separabilidade

*as classes se misturam?*

- Existe fronteira linear? Quanto os pontos das duas classes se tocam?
- **Medidas:** erro do classificador linear e N1

# Variação de conceito

- Captura a **rugosidade do conceito alvo** no espaço de instâncias
- Espaço rugoso significa que pequenas variações nos atributos **mudam o rótulo**
- Define-se  $\delta(e_i, e_j) = 0$  se dois exemplos vizinhos têm a mesma classe e **1** caso contrário
- Comparam-se pares de exemplos que diferem em apenas um atributo; para cada par, registra-se 0 se as classes são iguais e 1 se são diferentes. A média desses valores indica a rugosidade do conceito.

**Leitura:** valor perto de 0 indica conceito suave, com poucos vizinhos discordando. Perto de 0,5 os rótulos são praticamente aleatórios entre vizinhos — cenário ruim para kNN e para quase todo o resto.

# Não-linearidade: um teste com dados inventados

- 1 Escolha dois exemplos da mesma classe**  
Pegue pares de pontos que pertencem à mesma classe
- 2 Interpole entre eles**  
Gere um novo ponto por interpolação linear com coeficientes aleatórios — e atribua a ele a mesma classe
- 3 Treine no original, teste nos inventados**  
O classificador é treinado no conjunto original e avaliado apenas nesses pontos sintéticos
- 4 A taxa de erro é a medida**  
Erro alto significa que a fronteira é **rugosa**: ela corta o meio do caminho entre dois pontos da mesma classe

**Duas versões:** com um classificador linear e com 1NN. É o mesmo procedimento, trocando só o classificador — e os dois erros juntos dizem se a dificuldade é de forma da fronteira ou de ruído local.

# F1 — razão discriminante de Fisher

$$F1_i = ( \mu_1 - \mu_2 ) / ( \sigma_1 - \sigma_2 )$$

$\mu_1, \sigma_1$ : média e desvio padrão do atributo  $x_i$  na classe 1 ·  $\mu_2, \sigma_2$ : idem para a classe 2

- Mede, **atributo por atributo**, o quanto as duas classes estão afastadas em relação ao seu espalhamento
- Médias distantes somadas a desvios pequenos indicam **boa separação** naquele atributo, ou seja, problema mais fácil
- O valor final do dataset é o **máximo entre todos os atributos**
- Usa-se o máximo porque **basta um atributo bom**: se existe uma dimensão em que as classes se separam, o problema já é tratável

## F2 — volume da região de sobreposição

- Para cada atributo, delimita-se a faixa de valores de cada classe, do mínimo ao máximo
- A **região de sobreposição** é o trecho em que as duas faixas coincidem
- Mede-se o comprimento dessa região e divide-se pela **amplitude total** das duas classes — isso normaliza o valor
- O valor do dataset combina os atributos, por produto ou mínimo conforme a implementação

| Valor    | Significado                                                                          |
|----------|--------------------------------------------------------------------------------------|
| $F2 = 0$ | existe atributo sem nenhuma sobreposição: separável por hiperplano ortogonal ao eixo |
| $F2 > 0$ | há sobreposição em todas as dimensões; quanto maior, mais ambíguo                    |

# F3 e F4 — eficiência dos atributos

Ambos perguntam: **quantos exemplos um atributo consegue separar?** Mudam em quem responde — um atributo sozinho ou todos em conjunto.

## F3 · eficiência máxima individual

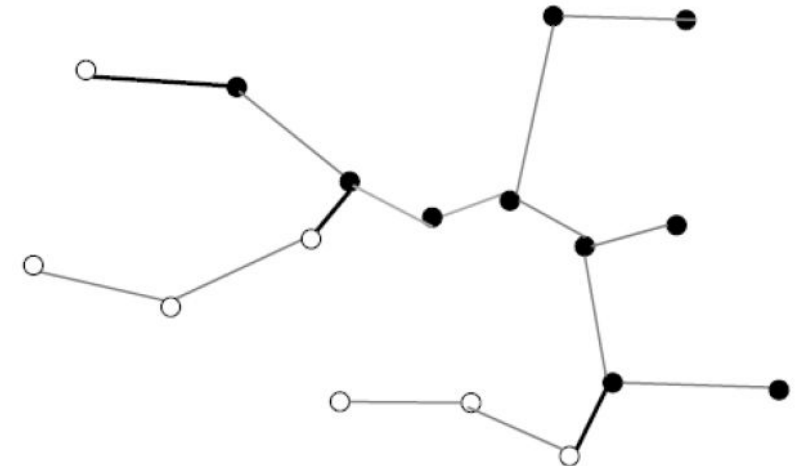
- Para cada atributo, identifica-se a região de sobreposição, como no F2
- Conta-se quantos exemplos ficam **fora** dela
- Eficiência = exemplos fora dividido pelo total de exemplos
- F3 é o **máximo** entre todos os atributos
- **Alto:** existe um atributo que já resolve boa parte do problema sozinho. **Baixo:** nem o melhor atributo separa quase nada

## F4 · eficiência coletiva

- Começa pelo atributo mais eficiente e **remove** os exemplos que ele já separou
- Sobre o resíduo, escolhe-se o próximo atributo que melhor separa
- Repete até acabarem atributos ou exemplos
- F4 é a proporção total de exemplos "limpos" pelo processo
- **Alto:** os atributos se complementam. **Baixo:** dataset genuinamente complexo

# N1 — fração de pontos na fronteira

- **Separabilidade linear:** a medida mais simples do grupo é a **taxa de erro de um classificador linear**
- **N1** usa a minimum spanning tree, ou árvore geradora mínima: conecta todos os pontos com a menor soma de distâncias possível e sem ciclos
- A MST conecta pontos **ignorando a classe**
- Contam-se as **arestas que ligam classes opostas**; os pontos nelas são os pontos de fronteira
- N1 é a fração desses pontos sobre o total



|                |                                                          |
|----------------|----------------------------------------------------------|
| N1 $\approx$ 0 | classes em "ilhas" separadas; problema fácil             |
| N1 $\approx$ 1 | alto entrelaçamento: fronteira não linear ou muito ruído |

### PARTE 3

# Quando a tarefa base muda

Regressão, séries temporais e agrupamento

# Regressão: o alvo virou numérico

Todo meta-atributo que envolve a variável alvo precisa ser reescrito.  
Os que envolvem só os atributos podem ser reaproveitados.

## Uma variável

- **Coeficiente de variação do alvo:**  $\sigma(y) / \mu(y)$
- **Número de outliers** do alvo  $y$
- **Densidade dos dados:**  $n / p$  — mesmo conceito visto em classificação

## Duas variáveis • correlação

- $q(x_i, y)$  — atributo com o alvo
- $q(x_i, x_j)$  — atributo com atributo
- Derivadas importantes: **q máximo** e **q médio** entre atributos e alvo

**Substituição-chave:** onde em classificação a pergunta era "esse atributo separa as classes?", em regressão ela vira "esse atributo se correlaciona com o alvo?".  $q$  máximo alto sugere que um único atributo já dá boas previsões.

# Complexidade e suavidade em regressão

| Medida                                                                     | Como é obtida                                                                                                                                                      |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Eficiência máxima individual (adapta o F3)</b>                          | Para cada atributo, menor número de exemplos a remover até que $q(x_i, y)$ ultrapasse um limiar. Converte-se em proporção; a medida é a mínima entre os atributos. |
| <b>Eficiência coletiva (adapta o F4)</b>                                   | Iterativamente: escolhe o atributo de maior correlação, faz um ajuste linear, elimina exemplos de resíduo pequeno. Mede a proporção que sobra ao final.            |
| <b>Similaridade do alvo entre vizinhos (adapta a variação de conceito)</b> | Constrói uma MST no espaço de entrada com pesos euclidianos e mede a distância média entre os valores de alvo conectados.                                          |
| <b>Similaridade dos atributos para alvos parecidos</b>                     | O complemento da anterior: exemplos com $y$ parecido têm atributos parecidos?                                                                                      |
| <b>Erro de um regressor 1NN (adapta o landmarker 1NN)</b>                  | Mesma ideia do landmarker de classificação, com métrica adequada a regressão, por exemplo MSE.                                                                     |
| <b>Resíduos de um regressor linear</b>                                     | Média dos resíduos absolutos e variância dos resíduos. Valores pequenos indicam problema mais simples.                                                             |

# Séries temporais: a ordem passa a importar

Aqui as instâncias **não são independentes** — existe uma ordem relevante. Lemke e Gabrys (2010) dividem as medidas em quatro grupos:

## Estatísticas gerais

- Antes de tudo, **destendenciona-se** a série com regressão polinomial
- Comprimento; desvio padrão da série destendenciada; assimetria; curtose
- **Tendência** =  $\text{std}(\text{série}) / \text{std}(\text{série destendenciada})$
- Pontos de virada; mudanças de passo; não-linearidade

## Domínio da frequência

- Derivadas de uma FFT aplicada à série
- Frequências dos três maiores valores
- Valor máximo: o componente sazonal mais forte
- Número de picos com ao menos 60% do máximo

## Autocorrelação

- Informam sobre **estacionaridade e sazonalidade**
- Autocorrelação nos lags 1 e 2
- Autocorrelação parcial nos lags 1 e 2
- Parcial no lag 7 ou 12: sazonalidade semanal ou mensal

# Agrupamento: sem variável alvo

Sem  $y$ , some metade do repertório — e é preciso construir descrições a partir das distâncias

## Baseadas em modelo · o vetor de distâncias

- Define-se o vetor  $\mathbf{d}$  com as distâncias euclidianas entre **todos os pares** de instâncias
  - O vetor é normalizado no intervalo  $[0, 1]$  e então caracterizado estatisticamente
- 
- **1. Medidas simples** — média, variância, desvio padrão, assimetria e curtose de  $\mathbf{d}$
  - **2. Histograma** — 10 bins de tamanho igual; cada bin vira um meta-atributo com a porcentagem de valores que caem nele
  - **3. Distribuição alternativa** — primeiro convertem-se os valores em z-scores, depois repete-se o histograma

# Agrupamento: sem variável alvo

Sem y, some metade do repertório — e é preciso construir descrições a partir das distâncias

## Simple, estatísticas e informação

- Só sobram as que envolvem **variáveis independentes**
- Ferrari e de Castro (2015) usam um subconjunto apropriado das de classificação
- Pimentel e de Carvalho (2019) descrevem a **distribuição de correlações de rank entre exemplos** — não entre atributos

## Baseadas em performance

- Usam **medidas de validação interna** após rodar K-means com configuração fixa
- Coeficiente de silhueta, índice de Davies-Bouldin
- Assim landmarks e landmarks por subamostragem se adaptam ao domínio

PARTE 4

# Engenharia de meta-atributos

Derivar mais, e depois cortar

# O problema da largura variável

- Assimetria, curtose, correlação — todas são calculadas **por atributo**
- Um dataset com 4 atributos gera 4 valores de assimetria; um com 300 gera 300
- Mas o vetor de meta-atributos precisa ter o **mesmo número de posições** para todos os datasets
- Solução usual: **agregar** com uma medida de sumário, tipicamente a média
- **E aí se perde informação: dois datasets muito diferentes podem ter a mesma média**

dataset A - assimetrias



dataset B - assimetrias



---

*mesma media, distribuicoes opostas*

**Solução de Kalousis e Theoharis (1999):** agregar por histograma. Em vez da média da assimetria, três meta-atributos: quantos atributos têm assimetria abaixo de 0,2, entre 0,2 e 0,4, e acima de 0,4. A largura fica fixa e o formato da distribuição sobrevive.

# Gerar um conjunto sistematicamente completo

Muitos trabalhos partem de um conjunto **incompleto** por dois motivos recorrentes:

- Não incluem **todos os grupos** de meta-atributos — ficam só nos simples e estatísticos, por exemplo
- Usam **uma única medida de agregação**, quase sempre a média

**Receita sistemática:** para cada medida por atributo, aplicar todas as agregações disponíveis — média, desvio padrão, mínimo, máximo, Q1, Q2, Q3 e as medidas de histograma.

- Conjunto mais completo **supera** o mesmo sistema com o conjunto inicial menor
- O preço é um espaço de meta-atributos muito maior — o que torna a **seleção** obrigatória, e não opcional

# Projeção e fatoração: features latentes

## Projeção

- PCA projeta os meta-atributos num espaço de baixa dimensão formado pelas componentes principais (Smith-Miles et al., 2014)
- Qualquer técnica de redução de dimensionalidade serve — o objetivo é o requisito 4
- Custo: as componentes deixam de ser interpretáveis

## Fatoração de matriz

- Fusi et al. (2018) aplicam **fatoração matricial probabilística** à matriz de performances  $P$  (algoritmos  $\times$  datasets)
- Cada célula de  $P$  é a performance de um algoritmo em um dataset
- A PMF decompõe  $P$  em duas matrizes de espaço latente,  $X$  e  $Y$

$X$  — mapeia cada dataset para um espaço latente: um vetor que descreve as "propriedades intrínsecas" que fazem os algoritmos se comportarem daquele jeito.

$Y$  — mapeia cada algoritmo para o mesmo espaço: descreve que tipo de dado aquele algoritmo "gosta".

# Seleção de meta-atributos

Meta-datasets têm poucas instâncias. Muitos meta-atributos levam a sobreajuste e a previsões ruins em datasets novos.

## Duas estratégias

- **A priori:** incluir apenas medidas que se espera relevantes, com base em conhecimento de especialista sobre o problema
- **A posteriori:** incluir o máximo possível e aplicar um método de seleção de atributos
- Todorovski et al. (2000): a **eliminação reversa** — remove um atributo por vez e avalia o impacto — melhora a performance, o que se atribui à remoção de atributos ruidosos

# Caracterização dinâmica (iterativa)

E se, em vez de escolher os meta-atributos de antemão, o sistema fosse **coletando conforme precisa**?

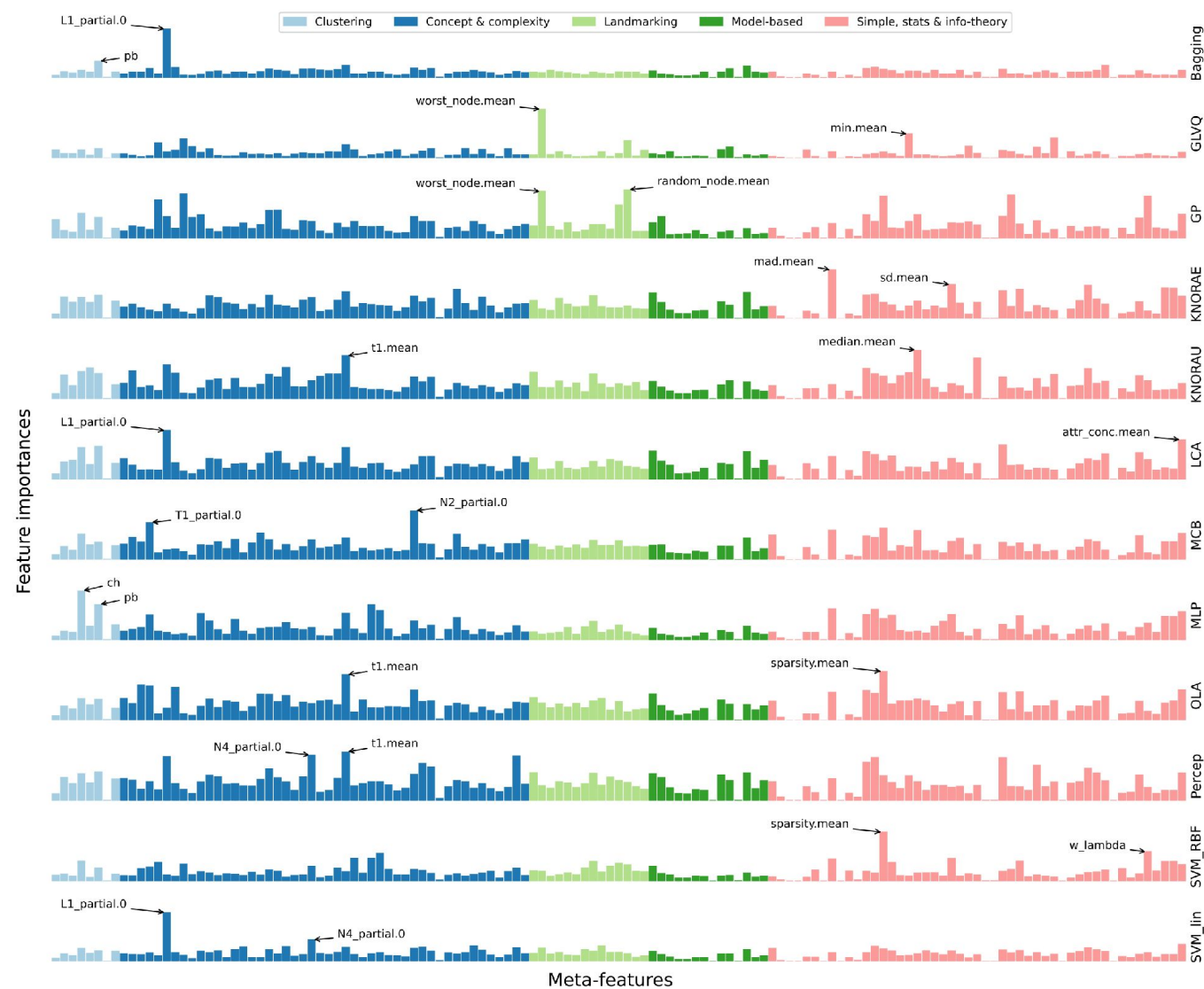
- Faz sentido quando **obter o meta-atributo custa caro** — landmarks por subamostragem, por exemplo
- Cenário: decidir se o algoritmo A ou B serve para o dataset alvo. Um teste em amostra pequena já dá alguma informação
- Mais amostras dão mais informação — mas se a decisão já pode ser tomada, **não há por que continuar gastando**
- A cada passo o sistema pergunta: o conjunto atual é suficiente? Se não, **qual extensão traz mais informação pelo menor esforço?**
- A resposta vem dos **metadados existentes**: o que aconteceu em situações parecidas no passado

A caracterização deixa de ser um passo de pré-processamento e vira parte da **busca**. O plano de experimentos é construído gradualmente, usando resultados de outros datasets e do próprio dataset alvo.

# Quais meta-atributos realmente pesam

- Ao treinar algoritmos **baseados em árvore** no meta-nível, ganha-se de brinde o vetor de **importâncias das meta-features**
  - Amorim et al. (2024) usam meta-aprendizagem para recomendar **técnicas de normalização** em classificação — um meta-modelo por classificador base
  - O resultado é um **perfil de importância diferente para cada classificador base**
- 
- SVM linear e Bagging dependem fortemente de L1\_partial — separabilidade linear, o que faz sentido
  - GLVQ e GP dependem de landmarks (worst\_node, random\_node)
  - OLA e SVM-RBF pesam sparsity — métodos locais sofrem com dados esparsos
  - Nenhuma família domina sozinha: o perfil muda conforme o algoritmo base

# Importâncias por classificador base



# Meta-atributos específicos ao par de algoritmos

A proporção de atributos contínuos discrimina bem entre Naive Bayes e kNN — e não discrimina quase nada entre Naive Bayes e um aprendiz baseado em regras.

- Sabe-se que kNN aproveita melhor atributos contínuos que NB. Já entre NB e regras, ambos têm dificuldade com contínuos — e o meta-atributo perde poder discriminativo
- Daí a estratégia: transformar seleção de algoritmos em **vários problemas binários** (A ou B?) e usar um **conjunto de meta-atributos diferente para cada par**
- Sun e Pfahringer (2013) treinam um classificador de regras por par; as regras geradas viram, elas mesmas, novos meta-atributos
- Quando os algoritmos são **muito parecidos**, é preciso criar meta-atributos que capturem as nuances entre eles
- Caso extremo: o **mesmo algoritmo com hiperparâmetros diferentes**. Soares e Brazdil (2006) mostram que meta-atributos derivados das **matrizes de kernel** superam os gerais na escolha do kernel de SVM

**Representação:** a maioria usa um vetor de tamanho fixo, mas há abordagens relacionais (ILP) que representam, por exemplo, a assimetria de cada atributo separadamente — sem agregação.

# Similaridade entre datasets

É aqui que os meta-atributos são efetivamente consumidos: encontrar os datasets do passado mais parecidos com o dataset alvo, e reaproveitar o que funcionou neles.

## META-ATRIBUTOS

$$Dist(d_{novo}, d_i) = \sum_n \frac{|f_{novo,n} - f_{i,n}|}{\max(f_n) - \min(f_n)}$$

- A similaridade sai da distância: **Sim = 1 - Dist**
- Cada dataset é um vetor; é literalmente o que o **kNN** faz ao buscar os k vizinhos mais próximos
- Outras métricas servem: euclidiana, Manhattan

## PERFORMANCE

$$Sim(d_{novo}, d_i) = \frac{p(a, d_{novo}) \cdot p(a, d_i)}{\|p(a, d_{novo})\| \|p(a, d_i)\|}$$

- Dois datasets são parecidos se os **mesmos algoritmos vencem** neles — parente próximo dos landmarks
- O denominador normaliza os valores para o intervalo [0, 1]; há também a variante baseada em **correlação**
- Limite prático: só entram algoritmos **já avaliados** no dataset novo

**1 Um dataset vira uma linha**

Meta-atributos são o que preenche essa linha; sem eles não há meta-nível

**2 Discriminar contra custar**

Os quatro requisitos entram em conflito: a medida mais informativa raramente é a mais barata

**3 Direto, por modelo, por desempenho**

As quatro famílias diferem no caminho pelo qual os dados são descritos

**4 Agregar é perder**

Toda medida por atributo exige agregação; histogramas perdem menos que a média

**5 Derivar muito, depois cortar**

Meta-datasets pequenos punem quem não seleciona

**6 Tudo serve à similaridade**

Na prática, o consumo final é: quais datasets do passado se parecem com este?